

Cloud Computing Tools

Deliverable 4: Containerizing Observability Services with Docker

Andersson Sarmiento Peña

Esteban Bejarano Caldas

Fredy Agatón Aguirre

Maestría en Analítica Aplicada, Universidad de La Sabana

Cloud Computing Tools

Hugo Franco, Ph.D.

Septiembre de 2026

1. Establecimiento de una conexión remota con la instancia de cómputo en la nube

La primera etapa de la actividad consistió en aprovisionar y acceder a una máquina virtual utilizando el servicio Amazon EC2 de Amazon Web Services (AWS). El objetivo fue disponer de un entorno de cómputo en la nube capaz de alojar el servicio de visualización requerido para la actividad.

La instancia EC2 fue aprovisionada utilizando una imagen de Amazon Linux 2023 y un tipo de instancia t2.micro. Para establecer la conexión remota se utilizó el par de claves cloud-docker-key.

La instancia fue asociada al grupo de seguridad grafana-ec2, el cual contenía reglas de entrada para permitir la conexión SSH mediante el puerto 22 y el acceso externo al servicio Grafana mediante el puerto 3000. Las reglas fueron restringidas a la dirección IP pública del equipo utilizado para realizar la actividad.

La instancia creada presentó la siguiente configuración:

- ID de instancia: i-017d03d0dfeff6a1a
- Tipo de instancia: t2.micro
- Sistema operativo: Amazon Linux 2023
- Grupo de seguridad: grafana-ec2
- Par de claves: cloud-docker-key
- Dirección IPv4 pública: 3.88.205.121

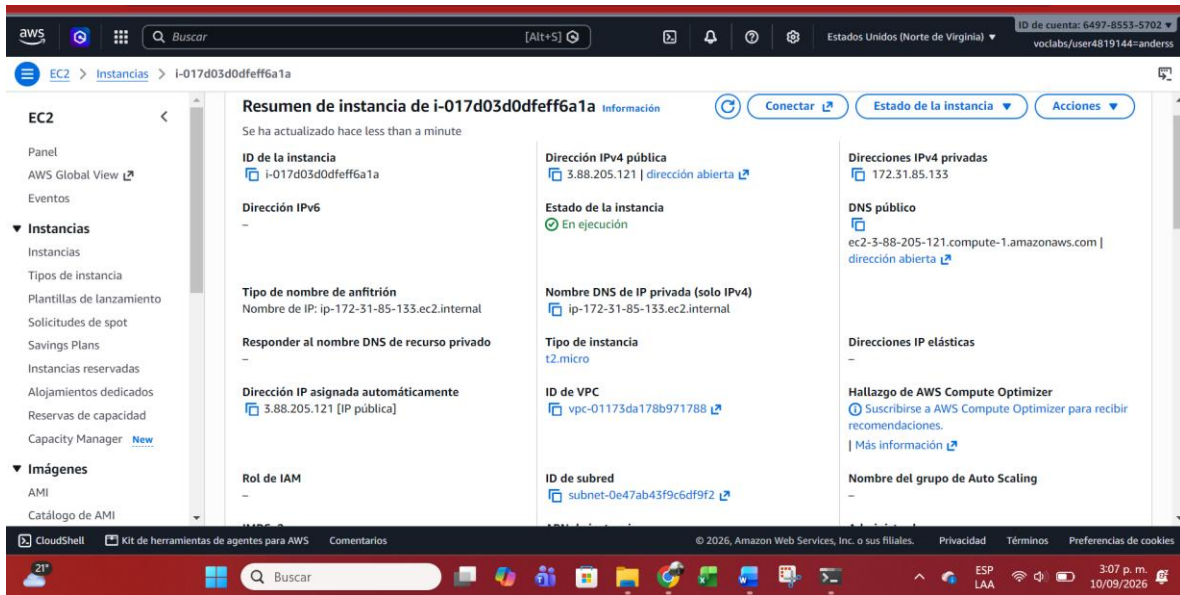


Figura 1. Instancia EC2 utilizada para el despliegue automatizado de Grafana.

Una vez que la instancia estuvo disponible y superó las comprobaciones de estado de AWS, se estableció una conexión remota mediante SSH utilizando el siguiente comando:

```
ssh -i .\cloud-docker-key.pem ec2-user@3.88.205.121
```

La conexión fue establecida correctamente, permitiendo acceder a la terminal Linux de la instancia y continuar con la configuración automatizada del entorno.

2. Aprovisionamiento automatizado de EC2 mediante boto3 y UserData

La segunda etapa consistió en automatizar la creación de la infraestructura EC2 mediante Python y la biblioteca boto3, que permite interactuar programáticamente con los servicios de AWS.

El script de automatización estableció una sesión con AWS utilizando las credenciales temporales proporcionadas por AWS Academy y configuró la región de trabajo us-east-1.

Posteriormente, se utilizó el recurso EC2 de boto3 para crear la instancia. Los principales parámetros de infraestructura fueron definidos directamente en el código, incluyendo la imagen de

Amazon Machine Image (AMI), el tipo de instancia, el par de claves, el grupo de seguridad y el bloque UserData.

La configuración utilizada para la creación de la instancia fue:

```
KEYPAIR_NAME = 'cloud-docker-key'
SG_IDS = ['sg-0b03a8ed27627b2ef']
instances = ec2.create_instances(
    ImageId='ami-025b6f0b1ac2ef9f7',
    MinCount=1,
    MaxCount=1,
    InstanceType='t2.micro',
    KeyName=KEYPAIR_NAME,
    UserData=user_data_script,
    SecurityGroupIds=SG_IDS,
)
```

El mecanismo principal de automatización fue el bloque UserData, enviado a la instancia durante su proceso de creación. Esto permitió que las tareas necesarias para configurar el servidor se ejecutaran automáticamente durante el arranque del sistema operativo.

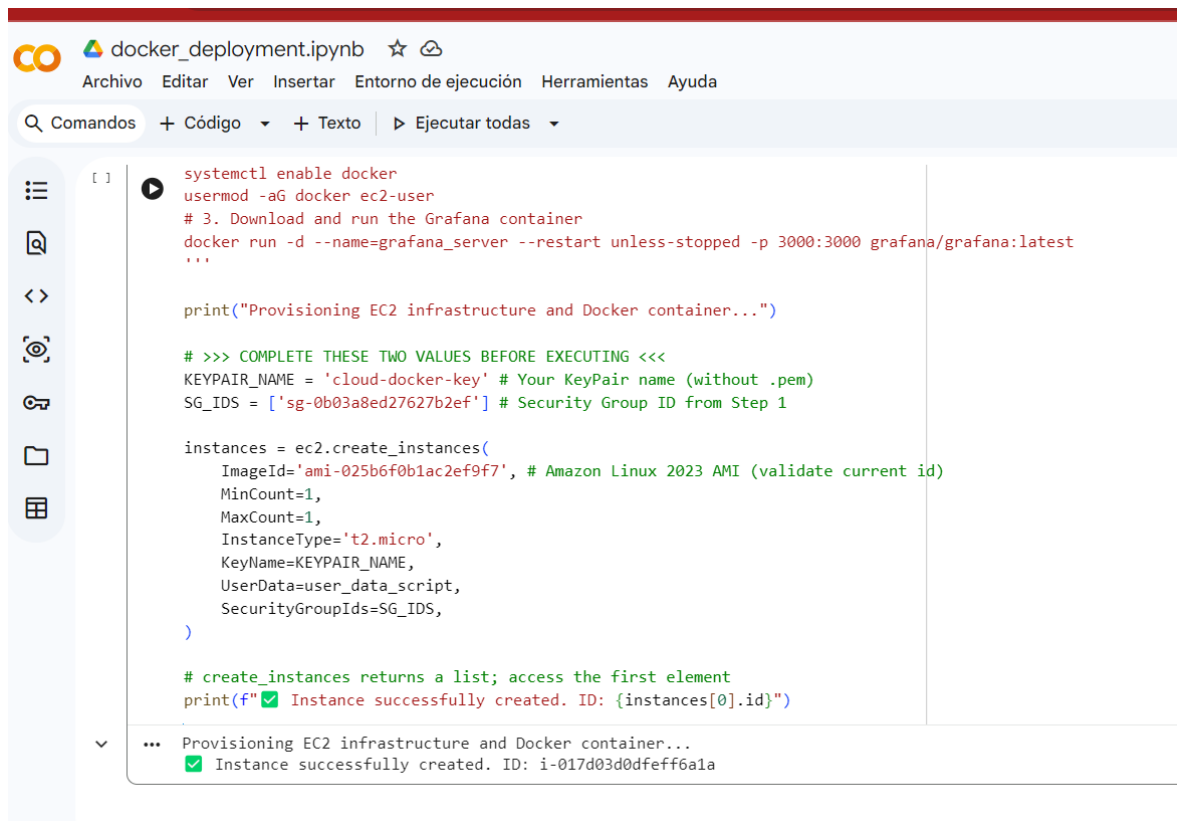
El script UserData realizó las siguientes acciones:

1. Actualización de los repositorios del sistema.
2. Instalación de Docker.
3. Inicio del servicio Docker.
4. Configuración de Docker para iniciar automáticamente.
5. Incorporación del usuario ec2-user al grupo Docker.
6. Descarga de la imagen de Grafana.
7. Creación y ejecución del contenedor de Grafana.
8. Exposición del servicio mediante el puerto 3000.

El bloque utilizado fue:

```
#!/bin/bash
dnf update -y
dnf install docker -y
systemctl start docker
systemctl enable docker
usermod -aG docker ec2-user
docker run -d \
  --name=grafana_server \
  --restart unless-stopped \
  -p 3000:3000 \
  grafana/grafana:latest
```

Esta implementación permitió aplicar principios de Infrastructure as Code (IaC), debido a que tanto el aprovisionamiento de la infraestructura como una parte importante de la configuración del servidor fueron definidos mediante código y ejecutados automáticamente.



```
docker_deployment.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Comandos + Código + Texto Ejecutar todas
[ ]
systemctl enable docker
usermod -aG docker ec2-user
# 3. Download and run the Grafana container
docker run -d --name=grafana_server --restart unless-stopped -p 3000:3000 grafana/grafana:latest
...

print("Provisioning EC2 infrastructure and Docker container...")

# >>> COMPLETE THESE TWO VALUES BEFORE EXECUTING <<<
KEYPAIR_NAME = 'cloud-docker-key' # Your KeyPair name (without .pem)
SG_IDS = ['sg-0b03a8ed27627b2ef'] # Security Group ID from Step 1

instances = ec2.create_instances(
    ImageId='ami-025b6f0b1ac2ef9f7', # Amazon Linux 2023 AMI (validate current id)
    MinCount=1,
    MaxCount=1,
    InstanceType='t2.micro',
    KeyName=KEYPAIR_NAME,
    UserData=user_data_script,
    SecurityGroupIds=SG_IDS,
)

# create_instances returns a list; access the first element
print(f"✅ Instance successfully created. ID: {instances[0].id}")

... Provisioning EC2 infrastructure and Docker container...
✅ Instance successfully created. ID: i-017d03d0dfeff6a1a
```

Figura 2. Creación exitosa de la instancia EC2 mediante el script de automatización desarrollado con boto3.

3. Instalación e inicialización de Docker Engine

Una vez creada la instancia EC2, el bloque UserData se ejecutó automáticamente durante el arranque del servidor. Como resultado, Docker Engine fue instalado y configurado sin necesidad de realizar manualmente el proceso de instalación desde la consola de AWS.

La correcta ejecución del servicio Docker fue comprobada mediante el siguiente comando:

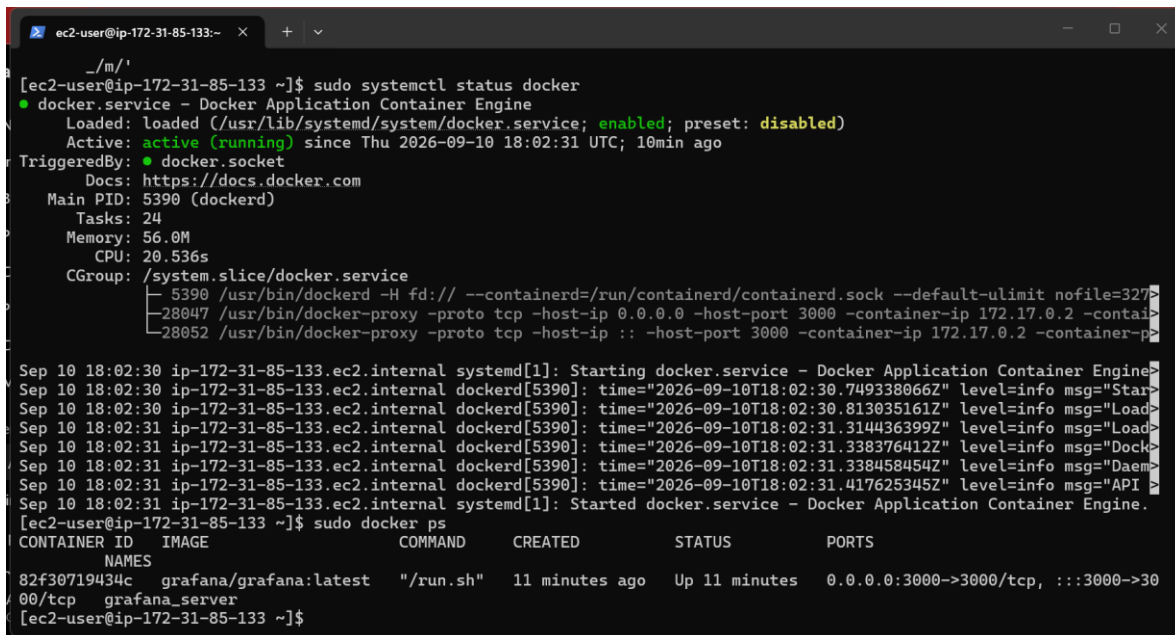
```
sudo systemctl status docker
```

El resultado mostró:

```
Active: active (running)
```

Esto confirmó que Docker Engine estaba instalado y ejecutándose correctamente.

Adicionalmente, el servicio aparecía como habilitado, lo cual permite que Docker se inicie automáticamente cada vez que el sistema operativo se reinicia.



```
ec2-user@ip-172-31-85-133:~$ sudo systemctl status docker
[ec2-user@ip-172-31-85-133 ~]$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 18:02:31 UTC; 10min ago
     TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 5390 (dockerd)
      Tasks: 24
     Memory: 56.0M
        CPU: 20.536s
    CGroup: /system.slice/docker.service
            └─ 5390 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768
              28047 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 3000 -container-ip 172.17.0.2 -container-port 3000
              28052 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 3000 -container-ip 172.17.0.2 -container-port 3000

Sep 10 18:02:30 ip-172-31-85-133.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine.
Sep 10 18:02:30 ip-172-31-85-133.ec2.internal dockerd[5390]: time="2026-09-10T18:02:30.749338066Z" level=info msg="Starting daemon"
Sep 10 18:02:30 ip-172-31-85-133.ec2.internal dockerd[5390]: time="2026-09-10T18:02:30.813035161Z" level=info msg="Loading containers"
Sep 10 18:02:31 ip-172-31-85-133.ec2.internal dockerd[5390]: time="2026-09-10T18:02:31.314436399Z" level=info msg="Loading containers: done"
Sep 10 18:02:31 ip-172-31-85-133.ec2.internal dockerd[5390]: time="2026-09-10T18:02:31.338376412Z" level=info msg="Docker daemon"
Sep 10 18:02:31 ip-172-31-85-133.ec2.internal dockerd[5390]: time="2026-09-10T18:02:31.338458454Z" level=info msg="Daemon is ready"
Sep 10 18:02:31 ip-172-31-85-133.ec2.internal dockerd[5390]: time="2026-09-10T18:02:31.417625345Z" level=info msg="API endpoint"
Sep 10 18:02:31 ip-172-31-85-133.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.

[ec2-user@ip-172-31-85-133 ~]$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
82f30719434c   grafana/grafana:latest   "/run.sh"           11 minutes ago   Up 11 minutes   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
grafana_server
```

Figura 3. Verificación del servicio Docker Engine ejecutándose correctamente en la instancia EC2.

Posteriormente, se verificó el contenedor desplegado mediante:

```
sudo docker ps
```

El resultado permitió comprobar que el contenedor de Grafana se encontraba activo. Se identificaron los siguientes elementos:

- Imagen: grafana/grafana:latest
- Nombre del contenedor: grafana_server
- Estado: Up
- Mapeo de puertos: 0.0.0.0:3000->3000/tcp

4. Configuración del aislamiento de puertos y acceso externo a Grafana

La cuarta etapa consistió en configurar la comunicación entre el contenedor Docker y la instancia EC2.

Para ello, el contenedor de Grafana fue ejecutado utilizando la opción:

```
-p 3000:3000
```

Esta configuración establece una correspondencia entre el puerto 3000 del contenedor y el puerto 3000 de la instancia EC2.

De esta manera, las solicitudes recibidas por la instancia en el puerto 3000 son dirigidas al servicio Grafana que se encuentra ejecutándose dentro del contenedor.

Adicionalmente, el grupo de seguridad grafana-ec2 contaba con una regla de entrada TCP para el puerto 3000. El acceso fue restringido a la dirección IP pública utilizada durante el desarrollo de la actividad, evitando una exposición innecesariamente amplia del servicio.

Una vez que Docker y Grafana estuvieron disponibles, se accedió al servicio mediante:

```
http://3.88.205.121:3000
```

La interfaz de inicio de sesión de Grafana fue cargada correctamente, demostrando que el servicio se encontraba disponible desde el navegador.

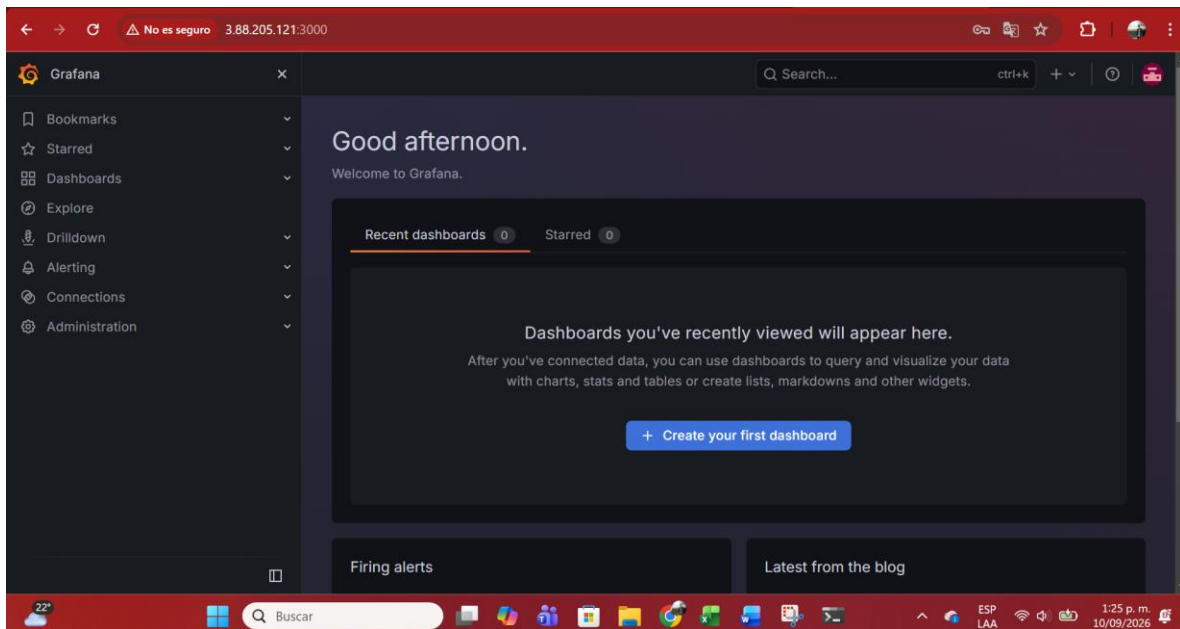


Figura 4. Acceso exitoso a la interfaz de Grafana mediante la dirección pública de la instancia EC2 y el puerto 3000.

Posteriormente, se inició sesión en Grafana utilizando las credenciales administrativas correspondientes para realizar la configuración de la fuente de datos y del dashboard.

5. Conexión de Grafana con Amazon RDS

La siguiente etapa consistió en conectar Grafana directamente con la base de datos PostgreSQL alojada en Amazon RDS.

La instancia RDS utilizada por el proyecto correspondió a:

```
db-cloud-analytics.cuhrcimjgwo.us-east-1.rds.amazonaws.com
```

En Grafana se configuró PostgreSQL como fuente de datos nativa utilizando los siguientes parámetros:

Parámetro	Configuración
Host	db-cloud-analytics.cuhrcimjgwo.us-east-1.rds.amazonaws.com:5432
Base de datos	postgres
Usuario	postgres
Modo TLS/SSL	require
Método TLS/SSL	File system path

La base de datos contenía la tabla:

```
datos_externos
```

Esta tabla almacenaba los registros de restaurantes obtenidos previamente mediante el flujo de datos del proyecto.

Después de ingresar los parámetros de conexión, se utilizó la opción Save & Test de Grafana.

La conexión fue validada exitosamente mediante el mensaje:

```
Database Connection OK
```

La configuración demuestra que Grafana utiliza Amazon RDS como fuente de datos nativa y que puede realizar consultas directamente sobre la información almacenada en PostgreSQL.

6. Consulta SQL y visualización de los datos

Una vez establecida la conexión entre Grafana y Amazon RDS, se creó un nuevo dashboard para visualizar la información almacenada en la base de datos.

La consulta SQL utilizada fue:

```
SELECT * FROM datos_externos;
```

La ejecución de esta consulta permitió recuperar los registros almacenados en la tabla datos_externos.

Entre los campos disponibles se encontraron:

- name

- address
- categories
- latitude
- longitude
- distance

La consulta permitió comprobar que Grafana podía acceder correctamente a los datos almacenados en Amazon RDS.

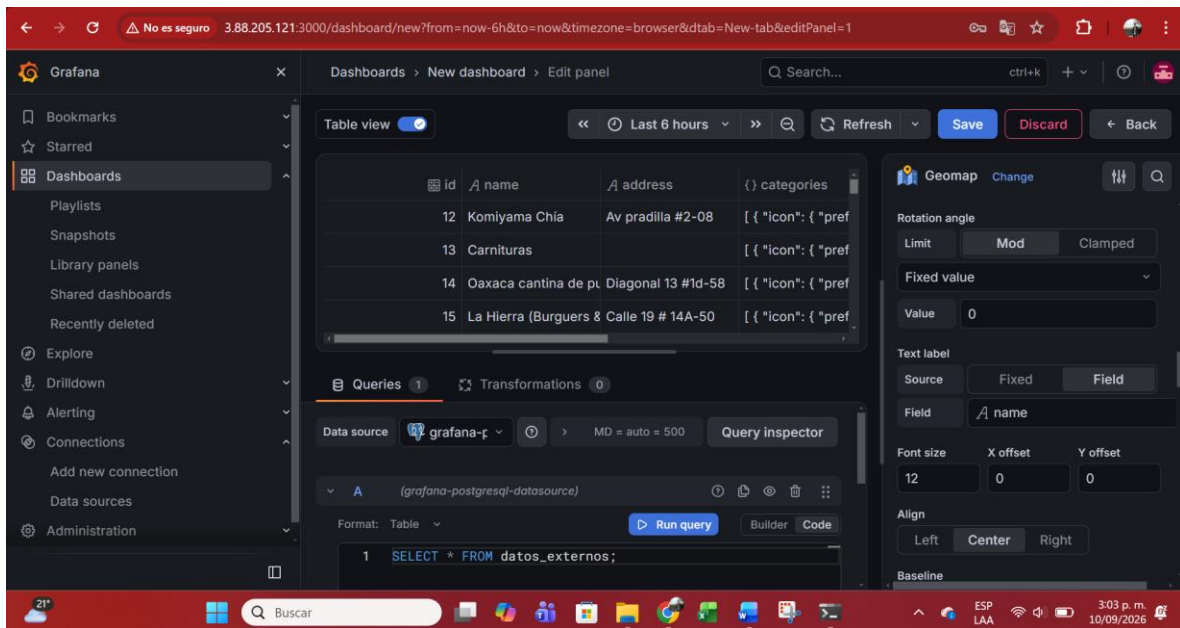


Figura 5. Consulta SQL ejecutada desde Grafana y registros recuperados desde la tabla datos_externos de Amazon RDS.

La evidencia obtenida demuestra la conexión funcional entre la capa de visualización y el almacén de datos PostgreSQL.

7. Visualización georreferenciada de los restaurantes

Debido a que el conjunto de datos contiene coordenadas geográficas, se configuró una visualización de tipo Geomap en Grafana.

La visualización utilizó los campos:

- latitude
- longitude

para determinar la ubicación geográfica de cada registro.

Grafana reconoció automáticamente las coordenadas mediante la configuración:

```
coords: latitude, longitude
```

Como resultado, los restaurantes fueron representados mediante marcadores sobre el mapa.

Además, se configuraron etiquetas para mostrar los nombres de los restaurantes asociados con los puntos geográficos.

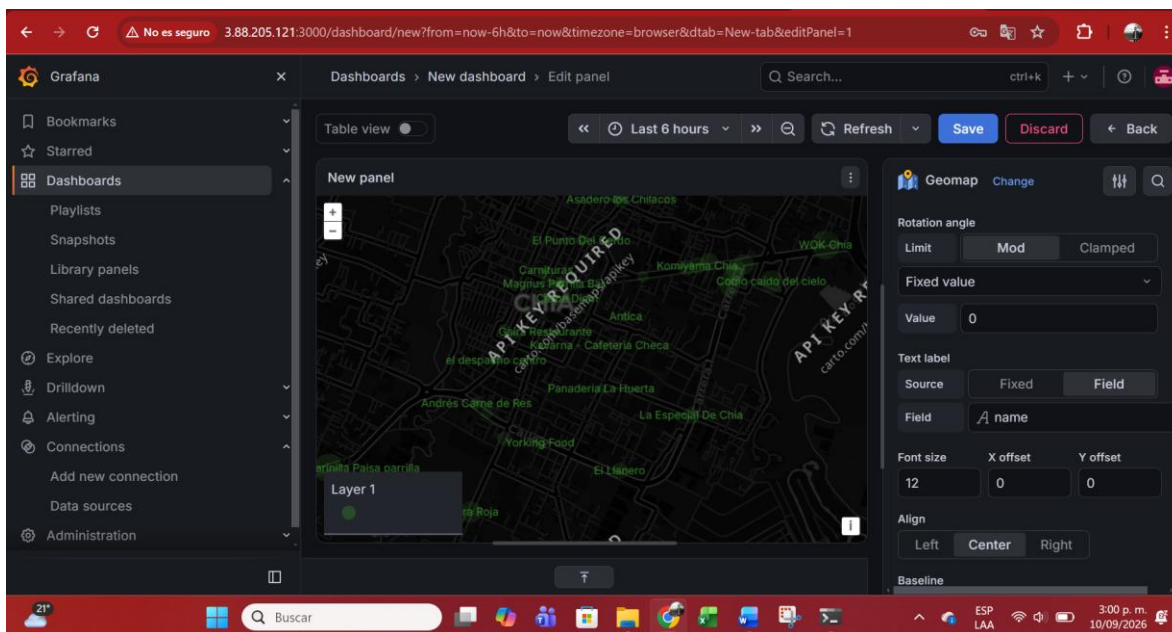


Figura 6. Visualización georreferenciada de los restaurantes mediante los campos de latitud y longitud almacenados en Amazon RDS.

La visualización permitió representar espacialmente los registros obtenidos mediante el pipeline de datos.

Durante la configuración del mapa base se presentó el mensaje “API Key Required”. Este mensaje corresponde al servicio utilizado para la capa base cartográfica y no afectó la consulta de datos ni la generación de los marcadores georreferenciados. Los puntos y nombres de los restaurantes fueron mostrados correctamente a partir de los datos almacenados en Amazon RDS.

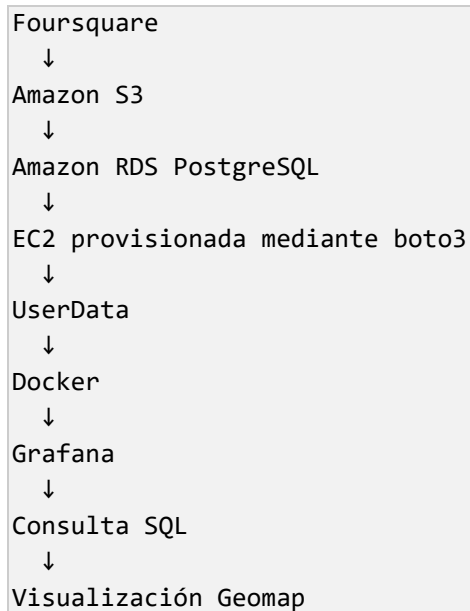
8. Dashboard interactivo resultante

Los componentes desarrollados durante la actividad fueron integrados en un dashboard de Grafana denominado:

Dashboard Restaurantes Foursquare - RDS

El dashboard constituye la capa final de visualización del pipeline implementado.

La arquitectura general desarrollada puede representarse de la siguiente manera:



El dashboard permitió consultar la información almacenada en Amazon RDS y representarla mediante una visualización geográfica interactiva.

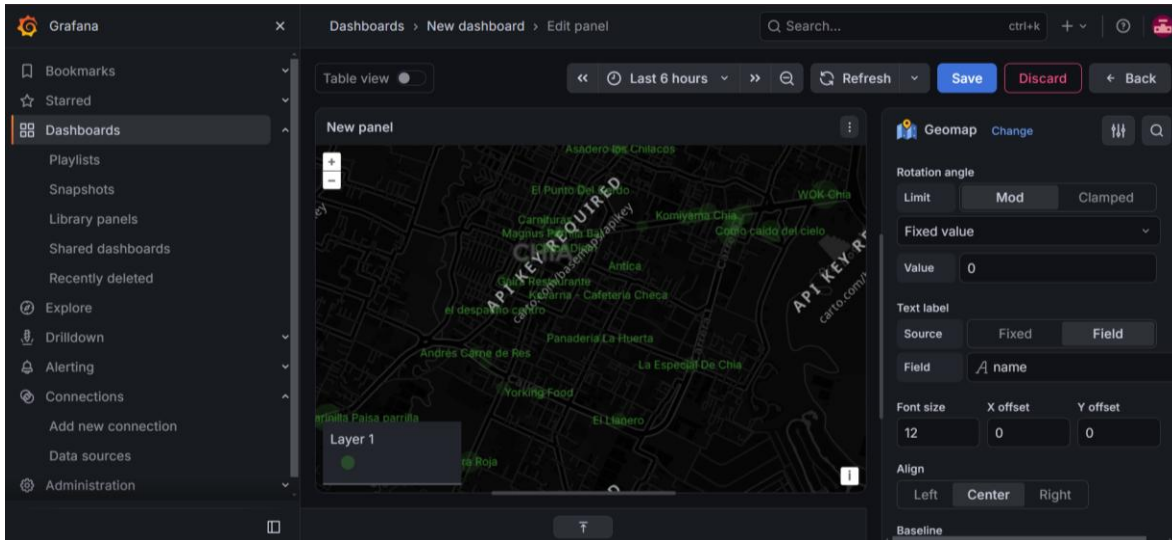


Figura 9. Dashboard final de Grafana con visualización georreferenciada de los restaurantes almacenados en Amazon RDS.

Visualización mediante gráfica circular

Como complemento a la representación geográfica, se implementó una visualización de tipo Pie Chart en Grafana para representar la distribución de los registros de restaurantes. La gráfica permite observar de manera proporcional la participación de cada establecimiento dentro del conjunto de datos almacenado en Amazon RDS PostgreSQL.

Cada segmento del gráfico representa un restaurante y su porcentaje corresponde a su participación dentro del total de registros consultados. Esta visualización permite complementar el análisis geográfico realizado mediante Geomap, proporcionando una perspectiva adicional sobre la composición de los datos.



Figura 10. Gráfica circular de distribución de restaurantes en Grafana.

Fuente: Elaboración propia a partir de los datos almacenados en Amazon RDS PostgreSQL y visualizados mediante Grafana.

Nota: Como la gráfica contiene numerosos restaurantes, los porcentajes individuales son pequeños y presentan valores cercanos entre sí. Por esta razón, esta visualización funciona principalmente como una representación complementaria de la composición del conjunto de datos, mientras que el mapa permite una interpretación más directa de la distribución espacial.

9. Gestión de recursos y estrategia FinOps

Como parte de las buenas prácticas de gestión de costos en la nube, la instancia EC2 fue utilizada únicamente durante el desarrollo, validación y recopilación de evidencias de la actividad.

Una vez comprobado el funcionamiento de:

- La instancia EC2.
- La automatización mediante boto3.

- El bloque UserData.
- Docker Engine.
- El contenedor Grafana.
- La exposición mediante el puerto 3000.
- La conexión con Amazon RDS.
- La consulta SQL.
- El dashboard.
- La visualización georreferenciada.

se procedió a terminar la instancia EC2.

La instancia terminada correspondió a:

i-017d03d0dfeff6a1a

La terminación de la instancia permitió evitar que el recurso de cómputo continuara generando costos después de finalizar la actividad.

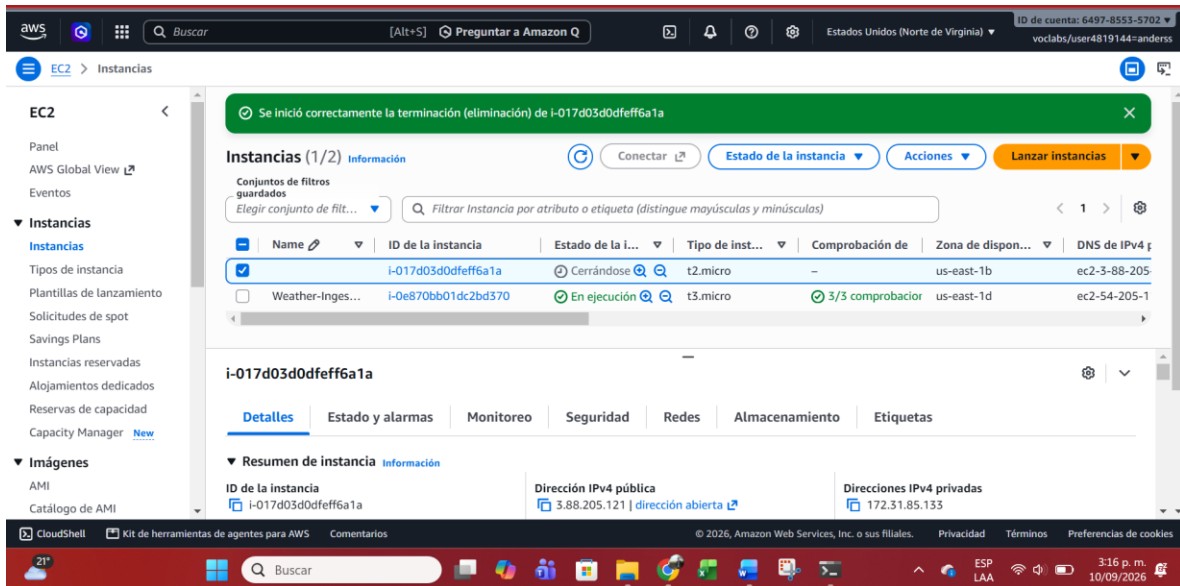


Figura 10. Terminación de la instancia EC2 después de finalizar la actividad como medida de control de costos bajo el enfoque FinOps.

Esta acción representa una aplicación práctica del principio de utilizar los recursos de nube durante el tiempo estrictamente necesario y liberarlos una vez cumplido su propósito.

10. Conclusiones

El desarrollo de la actividad permitió implementar satisfactoriamente un entorno automatizado de visualización de datos utilizando servicios de computación y almacenamiento en la nube.

En primer lugar, la infraestructura EC2 fue aprovisionada programáticamente mediante la biblioteca boto3, demostrando cómo es posible automatizar la creación de recursos de AWS mediante código.

En segundo lugar, se utilizó UserData para automatizar la configuración inicial de la instancia. Este mecanismo permitió actualizar el sistema, instalar Docker, iniciar el servicio y desplegar automáticamente el contenedor de Grafana durante el arranque de la máquina virtual.

En tercer lugar, Docker permitió ejecutar Grafana dentro de un contenedor aislado. La configuración -p 3000:3000 permitió exponer el servicio mediante el puerto 3000 de la instancia EC2, mientras que el Security Group permitió controlar el acceso externo.

En cuarto lugar, Grafana fue configurado para utilizar Amazon RDS PostgreSQL como fuente de datos nativa. La consulta:

```
SELECT * FROM datos_externos;
```

permitted comprobar que Grafana podía acceder directamente a los registros almacenados en la base de datos.

Finalmente, debido a que los registros contenían información geográfica, se implementó una visualización Geomap utilizando los campos latitude y longitude. Esto permitió representar espacialmente los restaurantes obtenidos a partir del flujo de datos del proyecto.

En términos generales, el flujo implementado puede resumirse como:

boto3 → EC2 → UserData → Docker → Grafana → Amazon RDS → SQL → Geomap

La actividad permitió integrar automatización de infraestructura, computación en la nube, contenedores, almacenamiento de datos y visualización interactiva dentro de un único flujo de trabajo.

Finalmente, la instancia EC2 fue terminada una vez finalizada la actividad, aplicando una estrategia básica de FinOps orientada a evitar el consumo innecesario de recursos y costos posteriores al desarrollo del taller.